



Qualcomm Technologies, Inc.

SW6100.LAW.1.0 Offload Framework

User Guide

80-74841-128 Rev. AA

February 13, 2026

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon® platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing™ products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to:
DocCtrlAgent@qualcomm.com.

Revision history

Revision	Date	Description
AA	February 2026	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

Contents

1 Offload framework overview	6
1.1 Offload mechanism and device modes	6
1.2 High-level architecture overview	7
1.3 Workflow overview	10
1.3.1 Flow of entering Tracker mode.....	10
1.3.2 Flow of exiting Tracker mode.....	10
2 Workflow for developing an Offload application	11
2.1 Set up the platform configurations	11
2.2 Integrate the Offload SDK.....	13
2.3 Prepare offload resources	13
2.4 Create and use Offload components.....	15
2.5 Handle interactive events.....	17
3 Offload SDK reference.....	18
3.1 General UI Offload resources	18
3.1.1 MultiScreen.....	18
3.1.2 Screen.....	19
3.1.3 Layout.....	19
3.1.4 Widget.....	20
3.1.5 Shape.....	23
3.1.6 Font	24
3.2 Offload component	25
3.2.1 WatchfaceComponent	27
3.2.2 StatusBarComponent	28
3.2.3 OffloadUIComponent.....	28
4 Offload sample application	29
4.1 Prerequisites.....	29
4.2 Prepare Offload resources.....	29
4.3 Create and use Offload components.....	34
4.4 Bind and handle events	37
A References	39
A.1 Data source set.....	39
A.2 Related documents	40

Figures

Figure 1-1 Mode transition	7
Figure 1-2 Offload Framework High Level Architecture	7
Figure 1-3 Tracker mode enter flow sequence	10
Figure 1-4 Tracker mode exit flow sequence	10
Figure 3-1 Offload component lifecycle	26
Figure 3-2 WatchFaceComponent usage core flow	27
Figure 3-3 OffloadUIComponent usage core flow	28
Figure 4-1 Sample Offload UI	29

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

Tables

Table 2-1 Fundamental Offload UI elements	14
Table 3-1 MultiScreen attributes	18
Table 3-2 Screen attributes	19
Table 3-3 RelativeLayout attributes	19
Table 3-4 RotationLayout attributes	20
Table 3-5 Common attributes of basic UI elements.....	20
Table 3-6 EventParams attributes	21
Table 3-7 Image attributes	22
Table 3-8 Text attributes	22
Table 3-9 Common shape style attributes	23
Table 3-10 Circle attributes	23
Table 3-11 Rectangle attributes	24
Table 3-12 Line attributes	24
Table 3-13 Triangle attributes	24

1 Offload framework overview

The Offload framework helps developers build always-on Offload applications with interactive capabilities on the SW6100.LAW platform.

This document guides you through developing your first sample offload application. It provides a comprehensive description about its architecture and underlying principles. Besides, it explains the overall workflow for developing an offload application, including:

- Set up the platform configurations
- Prepare offload resources
- Use the Offload SDK APIs
- Handle user interaction events

NOTE: This document mainly provides guidance on underlying principles. For a complete reference of interfaces involved during development, see the *SW6100.LAW.1.0 Offload SDK API Reference* document (80-74841-127).

1.1 Offload mechanism and device modes

Offload mentioned in this document is an always-on display capability during device mode transitions to save power. [Figure 1-1 Mode transition](#) shows the general mechanism of the Offload framework.

- When the device transitions from Active mode to Tracker mode, the companion processor takes over UI display while main processor is suspended, achieving reduced power consumption.
- In Tracker mode, the companion processor performs the following actions:
 - Independently draws the application interface based on the resources pre-delivered by the main processor
 - Detects basic user gestures such as taps or swipes.
- When the user presses the power key or a critical event occurs (for example, an incoming call or an alarm), the main processor will be woken up, and device transitions from Tracker mode to Active mode, where the main processor takes control of the display.

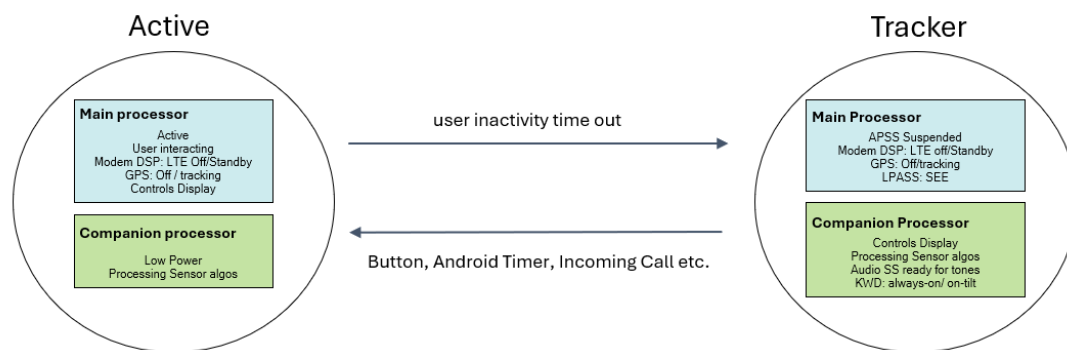


Figure 1-1 Mode transition

Overall, the core process of Offload includes the following actions:

- Define/parse/compose/convert offload resources.
- Send UI resources to the companion processor.
- Receive and deliver the interaction event from companion processor.
- Control the device mode transition.

1.2 High-level architecture overview

Figure 1-2 shows the high-level architecture of the offload framework.

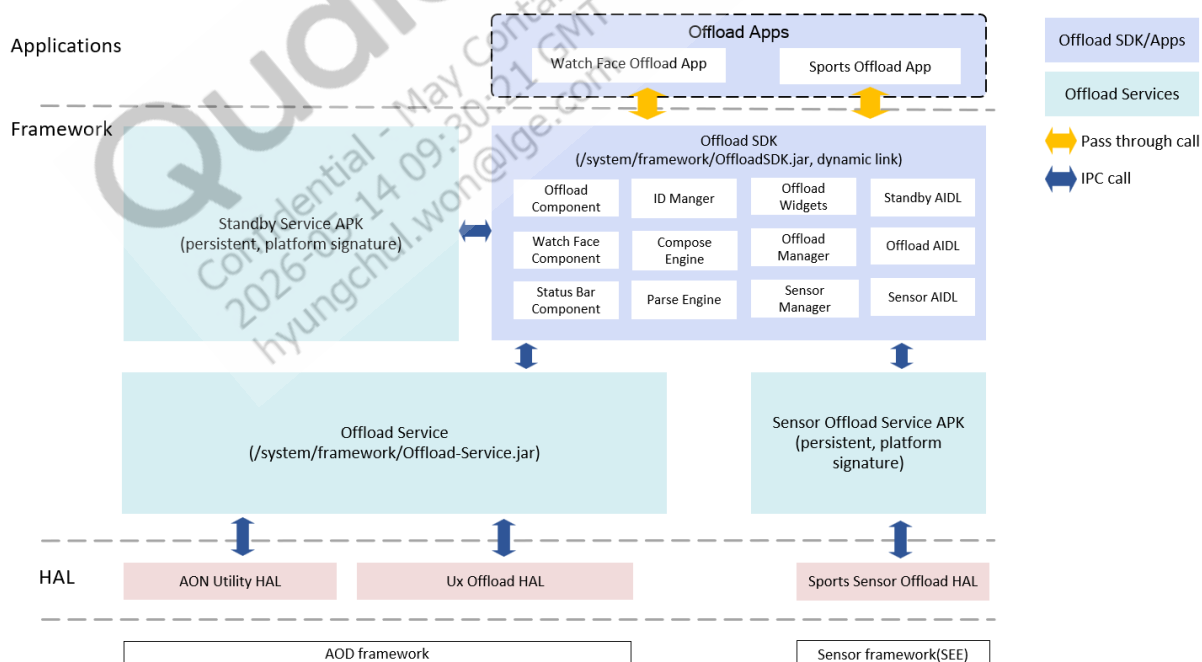


Figure 1-2 Offload Framework High Level Architecture

From top to bottom, the architecture includes the application layer, the Offload SDK, key system services, and the HAL layer.

1) Application

Refers to watch applications that support Offload, such as watch face apps and sports apps. You must use the interfaces provided by the Offload SDK to define resources within the app and register the app as a client of the framework, so that the system can invoke its resources while in Tracker mode.

2) Offload SDK

The Offload SDK encapsulates a set of APIs that allow applications to:

- Define and parse Offload UI resources (using XML layouts to describe).
- Register Offload components and communicate with system services.
- Send Offload resources to the system.
- Listen for device mode changes and handle callbacks.

3) Standby service APK

This persistent system application has the following core responsibilities:

- Monitor power-state change by extending Android's `DreamService` interface and further control the transition of display states.
- Receive offload resources from multiple offload applications, compose resources, and send composed offload resources to the offload service.
- Notify device mode changes of offload applications through the Offload SDK.

4) Offload service

This service module runs within the system server process and has the following functions:

- Provide AIDL APIs of offload functionality to `StandbyService` to send resources and commands.
- Control the display switch by extending Android's `DisplayOffloadSession` interface.
- Maintain mode-specific resource caches and convert them into the data format supported by HAL. After processing, resources are finally sent to HAL.
- Act as an event receiver when an interaction occurs on the specific offload widget under the Tracker mode. The event is returned from the companion processor through HAL callbacks into the Offload Service and then reported upward to the app through SDK.
- Serve as a bridge for context information synchronization, ensuring that specific UI tiles or data remain consistent during mode transitions.

5) Sensor offload service APK

This persistent system application has the following core functionalities:

- Provide AIDL APIs through the offload SDK to create and destroy sports sensor instances.
- Dispatch sensor event to sports sensor instances.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

1.3 Workflow overview

This information describes the interactions and timing sequences between the key modules.

1.3.1 Flow of entering Tracker mode

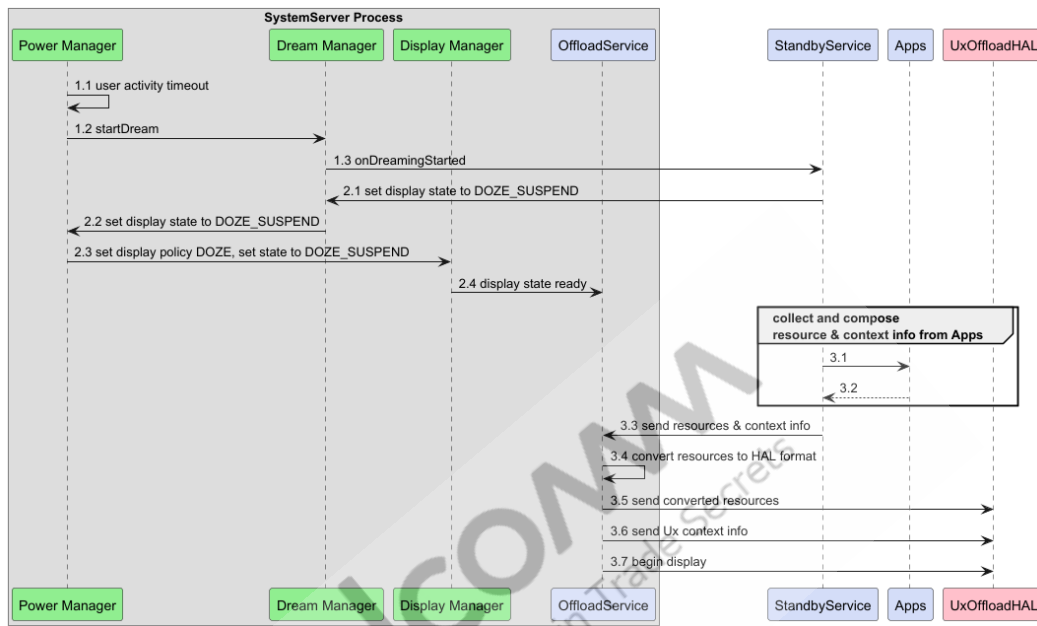


Figure 1-3 Tracker mode enter flow sequence

1.3.2 Flow of exiting Tracker mode

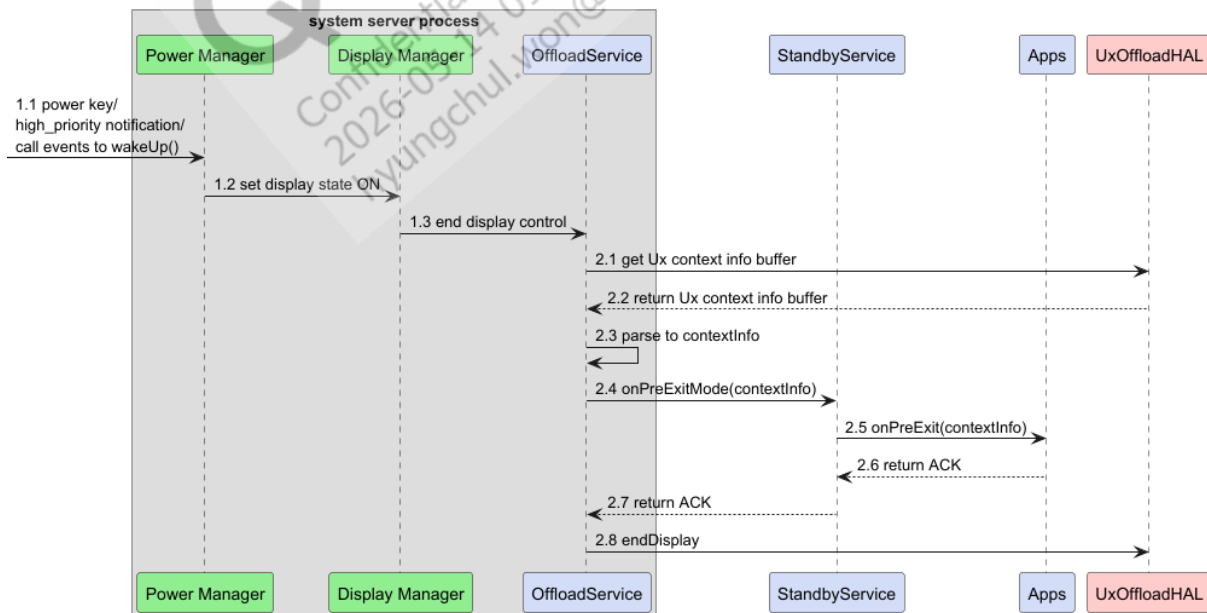


Figure 1-4 Tracker mode exit flow sequence

2 Workflow for developing an Offload application

This chapter walks you through the complete workflow required to build an Offload-enabled application, which includes the following steps:

1. [Set platform configurations.](#)
2. [Integrate the Offload SDK.](#)
3. [Prepare the necessary resource files.](#)
4. [Create and use Offload components within your application.](#)
5. [Handle interactive events.](#)

2.1 Set up the platform configurations

This information describes the basic platform configurations you need to set up before your development. The configuration consists of two types:

Configure the global switch

Enable the display offload functionality using the property values in the `system.prop` file. It acts as the master control of the Offload framework. By default, it is enabled. It acts as the master control of the Offload framework.

```
# Property to enable qti display offload
config.enable_qti_display_offload=true
```

Functional configuration

Customize the functional configurations using the settings defined within the framework's configuration resources.

NOTE: You can overlay these resources as needed to customize the specific behaviors.

Example code path:

```
/device/qcom/qssi_wear/overlay/frameworks/base/core/res/res/values/conf
ig.xml
```

```
<!-- Default dream service -->
<!-- This component will receive doze power state changes from system and then
control display state transition -->
```

```
<string name="config_dozeComponent">com.qualcomm.qti.standby/com.qualcomm.qti.standby.state.AlwaysOnService</string>

<!-- If the always on display mode is enabled.-->
<bool name="config_dozeAlwaysOnDisplayAvailable">true</bool>

<!-- Defines the ratio between the default brightness level in Trakcer mode
and the default brightness level in Active mode. It's used to scale the
screen brightness when the device enters Tracker mode, ensuring optimal
visibility while reducing power consumption.>
<fraction name="config_screenAutoBrightnessDozeScaleFactor">60%</fraction>

<!-- Remove the default dim process to achieve a smooth transition in screen
brightness.>
<integer name="config_maximumScreenDimDuration">0</integer>
<bool name="config_brightWhenDozing">true</bool>
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

2.2 Integrate the Offload SDK

As the Offload SDK is the direct dependency for the application, you must include it in the `bp` and `manifest` file.

1) `Android.bp`

```
android_app {
    name: "OffloadSample",
    srcs: ["src/**/*.java"],
    resource_dirs: ["res"],
    libs: [
        "UxOffloadSDK",
    ],
    platform_apis: true,
    optimize: {
        shrink: false,
    },
}
```

2) `AndroidManifest.xml`

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.Qualcomm.qti.offloadsample.watchface">

    <application
        android:stateNotNeeded="true"
        .....
        <uses-library android:name="UxOffloadSDK" />

    </application>
</manifest>
```

2.3 Prepare offload resources

The Offload framework uses XML files to construct the resources. This section describes the basic rules and fundamental elements. For detailed information, see [Section 3.1](#).

Based on the resources provided by Offload SDK and your actual requirements, you can then create the resource files as needed.

Fundamental elements can be categorized into the following types:

Table 2-1 Fundamental Offload UI elements

	Tags	Comments
MultiScreen	<MultiScreen>	Use this element when the app involves multiple screen page transitions. Once it's used, it must include multiple Screen tags as its child elements.
Screen	<Screen>	Use this element at the top level if MultiScreen is not used. It contains one or more layouts.
Layout	<RelativeLayout> <RotationLayout> ...	A layout contains one or more layouts or widgets or shapes.
Widget	<Image> <Text> ...	This is a fundamental element contained in the layout.
Shape	<Circle> <Rectangle> ...	This is a fundamental element contained in the layout.

The following is an example of a basic resource XML file. For details about the attributes, see [Section 3.1](#).

```

<MultiScreen
  name="multiscreen">
  <Screen
    name="analog_watchface"
    isActive="true"
    font="/system/fonts/RobotoStatic-Regular.ttf"
    compositionStrategy="LAYER_COMPOSITION">

    <Image
      name="background_image"
      src="@drawable/bg"
      offsetX="0"
      offsetY="0"
      needCompression="true">
      <WidgetEvents eventType="SINGLE_TAP" eventDestType="APPS"/>
    </Image>

    <RelativeLayout
      name="analog_hands_layout"
      offsetX="0"
      offsetY="0"
      width = "444"
      height ="444">
      <RotationLayout
        name="hands_rotation"
        pivotX="222"
        pivotY="222">

        <Image

```

```

        name="hour_hand_image"
        src="@drawable/hand_hour"
        offsetX="188"
        offsetY="92"
        angleExpr="time.hour12.deg">
</Image>

<Image
    name="min_hand_image"
    src="@drawable/hand_min"
    offsetX="188"
    offsetY="54"
    angleExpr="time.min.deg">
</Image>

<Image
    name="sec_hand_image"
    src="@drawable/hand_second"
    offsetX="188"
    offsetY="77"
    angleExpr="time.min.deg">
</Image>
</RotationLayout>
</RelativeLayout>
</Screen>
</MultiScreen>

```

NOTE: If there is no need to use **MultiScreen**, then **Screen** must be at the top to describe the resources.

2.4 Create and use Offload components

This information describes how to extend an Offload component for your application.

Prerequisites:

- You have set up the platform configurations and integrated the Offload SDK.
- You have created the XML definitions for the offload resources.

To implement the offload functionality, you need to perform the following operations:

- Parse resources
- Compose resources
- Send resources to the system
- Define how the application should behave when entering and exiting **Tracker mode** for seamless transition.

All the preceding operations are encapsulated as components and interfaces. You only need to inherit them and define the corresponding behaviors.

The Offload SDK provides multiple types of offload components for application development. You can choose the appropriate component according to your application type. For details, see [Section 3.2](#).

The following example code shows how to extend an Offload component using the common development and usage flow.

```
public class WatchFaceComponentImpl extends WatchFaceComponent {
    ...
    @Override
    public void onConnect() {
        // notify app connected with standbyService
    }

    @Override
    public void onDisconnect() {
        // notify app disconnected with standbyService
    }

    @Override
    public void onEnterMode(int mode) {
        // This method is called when device entered given mode.
        // handle logic when enter given devices modes,
        // such as stop to refresh watch face.
    }

    @Override
    public void onExitMode(int mode) {
        // This method is called when device exited given mode.
        // handle logic when exit given devices modes,
        // such as start to refresh watch face.
    }

    @Override
    void onPreEnterMode(int mode) {
        // This method is called when preparing to enter given mode.
    }

    @Override
    public void onPreExitMode(int mode, AppContext context,
                              IPreDrawCallBack callback) {
        // This method is called when preparing to exit given mode.
        // Note: must call callback.onPreDrawEnd() when draw is finished.
    }
}
```

```

@Override
public void onSendContextInfo(int mode) {
    // This method is called before entering given mode.
    // Request client to send current screen tiles or
    // data information.
}

@Override
public void onSendResource(int mode) {
    // This method is called just before entering given mode,
    // Request client to send runtime Offload resource.
}
}

```

2.5 Handle interactive events

This information shows how to handle interactive events with example code implementation.

Some resources often support simple interactive behaviors, such as **Image** and **Button**, which may respond to actions like tapping or long pressing. Those interactive behaviors are called **UxEvent**.

When the user performs interactions while in Offload mode, the system will relay these events back to the app, and the app needs to handle these events accordingly.

The following example code shows the core logic.

```

public class SampleClass implements IAppUxEventListener {

    // Define offload resource
    private AbstractOffloadResource mSampleBtn;

    private void init() {
        // Need to register listener when init
        mSampleBtn.registerAppUxEventListener(this);
    }

    @Override
    public void onAppUxEvent(
        AbstractOffloadResource abstractOffloadResource,
        AppUxEvent appUxEvent) {
        // This method is called when there are uxevents under offload
        // mode. Need to add logic to handle event according to
        // requirement.
    }
}

```

3 Offload SDK reference

This information includes details about Offload SDK for enabling richer and more specialized features:

- Essential UI resources
- Offload component: key principle and detail information

3.1 General UI Offload resources

As mentioned in [Section 2.3](#), offload resources are generally categorized into following types: MultiScreen, Screen, Layout, Widget, and Shape. This section describes their attributes and requests in detail, helping developers better understand and customize offload UI resources.

3.1.1 MultiScreen

`MultiScreen` describes a collection of multiple `Screen` resources. Use `MultiScreen` when an app contains several switchable tiles.

Table 3-1 MultiScreen attributes

XML attributes	Type	Request	Description
<code>name</code>	String	Mandatory	Indicates the name of the MultiScreen.
<code>isActive</code>	Boolean	Optional	Indicates whether the MultiScreen is effective. Default: <code>true</code> .
<code>compositionStrategy</code>	String	Mandatory	Specifies the composition strategy. Effective when <code>isActive</code> is true. Values: ▪ <code>NOT_COMPOSITION</code> – Don't compose status bar. ▪ <code>LAYER_COMPOSITION</code> - Compose status bar. ▪ <code>SCREEN_COMPOSITION</code> - Compose the multiple sub-screen resources. Default: <code>NOT_COMPOSITION</code>

3.1.2 Screen

`Screen` contains all the UI contents shown in a display screen. There are four defined attributes: `name`, `isActive`, `font` and `compositionStrategy`.

Table 3-2 Screen attributes

XML attributes	Type	Request	Description
<code>name</code>	String	Mandatory	Indicates the name of the Screen.
<code>isActive</code>	Boolean	Optional	Indicates whether the Screen is effective. Default: true.
<code>compositionStrategy</code>	String	Optional	Specifies the composition strategy. Effective when <code>isActive</code> is true. Value: NOT_COMPOSITION – Don't compose status bar. LAYER_COMPOSITION – Compose status bar. Default: NOT_COMPOSITION
<code>font</code>	String	Optional	Specifies the font filename for Screen. For details, see section 3.1.6. Default: NULL

3.1.3 Layout

a) RelativeLayout

`RelativeLayout` is a layout that describes the relative position to the parent and specifies the area, visibility and rotation angle according to attributes.

Table 3-3 RelativeLayout attributes

XML attributes	Type	Request	Description
<code>name</code>	String	Mandatory	The name of the <code>RelativeLayout</code> .
<code>isVisible</code>	Boolean	Optional	Specifies if this layout is visible. Value: true/false. Default: true. Effective only if <code>visibleExpr</code> = NULL
<code>width</code>	int	Optional	(Optional) Limits the width.
<code>height</code>	int	Optional	(Optional) Limits the height.
<code>offsetX</code>	int	Mandatory	The offset X relative to parent in pixels.
<code>offsetY</code>	int	Mandatory	The offset Y relative to parent in pixels.
<code>angle</code>	float	Optional	Specifies the rotation angle. Default: 0 Effective only if <code>angleExpr</code> = NULL.
<code>angleExpr</code>	String	Optional	Rotation angle DataSource(String). For details, see A.1 Data source set.

XML attributes	Type	Request	Description
visibleExpr	String	Optional	Visibility DataSource(String). Default: NULL For details, see A.1 Data source set
EventParams	Self-defined tag	Optional	Special resource type used to bind with interactive UI controls. Refer to Table 3-6 .

b) RotationLayout

`RotationLayout` is a layout that has a pivot to rotate around and collaborates with `RelativeLayout`. For example, `RotationLayout` specifies the pivot and `RelativeLayout` specifies the rotation angle. Detailed usage can refer to [Section 2.3](#).

Table 3-4 RotationLayout attributes

XML attributes	Type	Request	Description
name	String	Mandatory	The name of the RotationLayout.
pivotX	Float	Mandatory	The rotation pivot x relates with the parent layout.
pivotY	Float	Mandatory	The rotation pivot y relates with the parent layout.
EventParams	Self-defined tag	Optional	Special resource type used to bind with interactive UI controls. Refer to Table 3-6 .

3.1.4 Widget

A widget has common and element specific attributes. They are described as follows:

There are common basic attributes for different kinds of widgets and shapes as they are basic elements in screen.

Table 3-5 Common attributes of basic UI elements

XML attributes	Type	Request	Description
name	String	Mandatory	Indicates the name of the UI resource.
isVisible	Boolean	Optional	Indicates if this layout is visible. Values: true/false. Default: true. Effective only if <code>visibleExpr</code> = NULL
width	int	Optional	Limits the width.
height	int	Optional	Limits the height.
offsetX	int	Mandatory	The offset X relative to parent in pixels.

XML attributes	Type	Request	Description
offsetY	int	Mandatory	The offset Y relative to parent in pixels.
angle	float	Optional	Specifies the rotation angle. Default: 0. Effective only if angleExpr = NULL.
angleExpr	String	Optional	Rotation angle DataSource(String). For details, see A.1 Data source set
visibleExpr	String	Optional	Visibility DataSource(String). Default: NULL. For details, see A.1 Data source set
EventParams	Self-defined tag	Optional	Special resource type used to bind with interactive UI controls. For details, see Table 3-6 .

Some resources may involve user-interactive events, so we provide a special custom resource `EventParams` to describe them. In most cases, it will be used in widgets like Image or Button. But other types, even Layout, also support it.

Table 3-6 EventParams attributes

XML attributes	Type	Request	Description
eventType	int	Mandatory	The event type of the eventParams. Values: - TYPE_CLICK: 1 - TYPE_DOUBLE_CLICK: 2 - TYPE_TAP_AND_HOLD: 3 Default: TYPE_CLICK In the resource xml file, you must define it as "TYPE_CLICK".
eventDestType	int	Mandatory	The event destination type of the eventParams. Values: - TYPE_DEST_APPS: 1 the main processor will consume and deal with the event. - TYPE_DEST_LPI: 2 the companion processor will consume and deal with the event. Default: TYPE_DEST_APPS
wakeCapable	Boolean	Optional	If true, it requires waking up power. Default false.

a) Image

A UI element displayed as drawable image resources.

Table 3-7 Image attributes

XML attributes	Type	Request	Description
needCompression	Boolean	Optional	Specifies whether the image needs compression. Default: false.
imageType	int	Optional	Specifies the image format: 1: PNG 2: SVG
src	String	Mandatory	Indicates the path of image. Supports 2 types: - Asset type image: Use <fullpath> in XML. It is a fixed image. - Android-res type image: Use <@imageName> in XML. It loads different images according to different device configurations.

b) Text

A UI element displayed as text.

Table 3-8 Text attributes

XML attributes	Type	Request	Description
text	String	Optional	Text content. Effective if textExpr = NULL.
textExpr	String	Optional	Text content as DataSource(String). Default: NULL For details, see A.1 Data source set.
font	String	Optional	Font filename set for text. Default: NULL
format	String	Optional	Values: %s %d %f Default: %s
color	String	Optional	Specifies text color in Hex value #xxxxxxx. Supports ARGB and RGB. Default: #00000000.
fontSize	int	Mandatory	Font size.

XML attributes	Type	Request	Description
align	String	Optional	Specifies the alignment of text. Values: ▪ LEFT ▪ CENTER ▪ RIGHT Default: CENTER.

c) Button

A UI element displayed as a button. But there are no special attributes in Button, it follows the common attributes.

3.1.5 Shape

Besides common attributes mentioned in [Table 3-5](#), shapes also have some other common attributes representing the style parameters.

Table 3-9 Common shape style attributes

XML attributes	Type	Request	Description
borderWidth	Float	Mandatory	Indicates the border width.
color	String	Mandatory	Indicates the color of the resource.
isFilled	Boolean	Optional	If it is true, color fills the shape. ▪ Value: true or false. ▪ Default: true.
blendMode	String	Optional	How destination overrides source. Values: ▪ BL_SRC_OVER ▪ BL_DST_OVER ▪ BL_ADD Default: BL_SRC_OVER

a) Circle

Table 3-10 Circle attributes

XML attributes	Type	Request	Description
radius	Float	Mandatory	Indicates the radius.

b) Rectangle

Table 3-11 Rectangle attributes

XML attributes	Type	Request	Description
left	Float	Mandatory	x0.
top	Float	Mandatory	y0.
right	Float	Mandatory	x1.
bottom	Float	Mandatory	y1.
radius	Float	Optional	Radius of rectangle corners.

c) Line

Table 3-12 Line attributes

XML attributes	Type	Request	Description
point1X	Float	Mandatory	x0.
point1Y	Float	Mandatory	y0.
point2X	Float	Mandatory	x1.
point2Y	Float	Mandatory	y1.

d) Triangle

Table 3-13 Triangle attributes

XML attributes	Type	Request	Description
point1X	Float	Mandatory	x0.
point1Y	Float	Mandatory	y0.
point2X	Float	Mandatory	x1.
point2Y	Float	Mandatory	y1.
point3X	Float	Mandatory	x2.
point3Y	Float	Mandatory	y2.

3.1.6 Font

Font is a special attribute that specifies the TTF resources by the TTF filenames or file path names. You can specify two types of TTF resources for the font: the system TTF and the application TTF.

- System TTF is provided by the system and only TTF filenames needed to be defined in the attribute, such as, "DroidSansMono.ttf".
- Application TTF is preset in the asset folder that is the root directory for all TTF files.

To specify the application TTF, use "." to represent the asset folder. Both TTF path and name are needed, such as ". /font1.ttf" or ". /font/font2.ttf".

3.2 Offload components

The Offload SDK provides a set of components for offloading different types of UI resources. As described in [Section 2.4](#), the application must extend an Offload component to implement the functionalities.

3.2.1 Offload component types

The Offload SDK defines the following Offload components:

1) WatchFaceComponent

It is designed for offloading watch face UI resources. A watch face offload application extends this component. A watch face application is implemented in the wallpaper service.

2) StatusBarComponent

It is designed for offloading system status bar resource. Applications that show a system icon extend this component.

3) OffloadUIComponent

It is designed for offloading general UI resources other than watch faces and the status bar. Common offload applications (such as sports offload applications) extend this component.

3.2.2 Component lifecycle

All the Offload components share a common lifecycle, regardless of the specific component type.

Figure 3-1 illustrates the lifecycle of an Offload component.

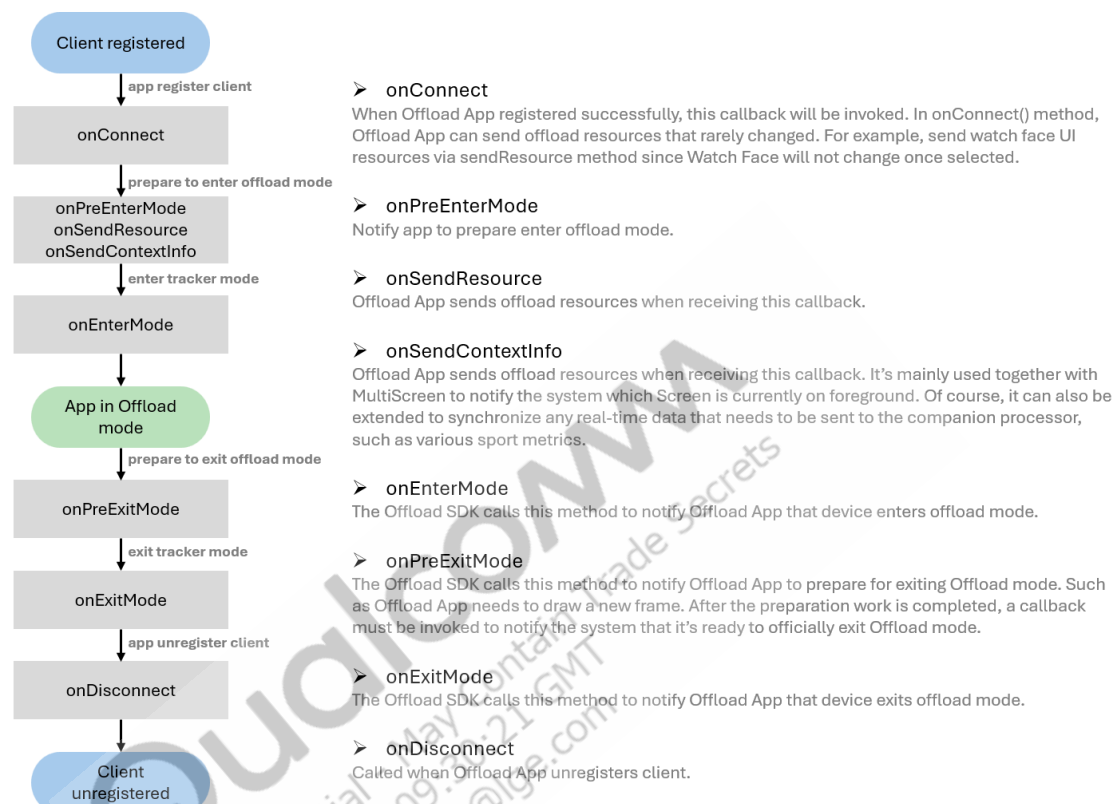


Figure 3-1 Offload component lifecycle

3.2.3 Component-specific usage flows

Although all Offload components share the same lifecycle, their usage flows differ, mainly in when and how UI resources are prepared and sent.

The following sections describe the typical usage flow for each Offload component.

3.2.3.1 WatchfaceComponent

`WatchFaceComponent` sends UI resources as part of its startup process. After the device boots up and the watch face application starts, the component prepares and sends the required resources to the Offload system.

Figure 3-2 shows the typical usage flow of `WatchFaceComponent`.

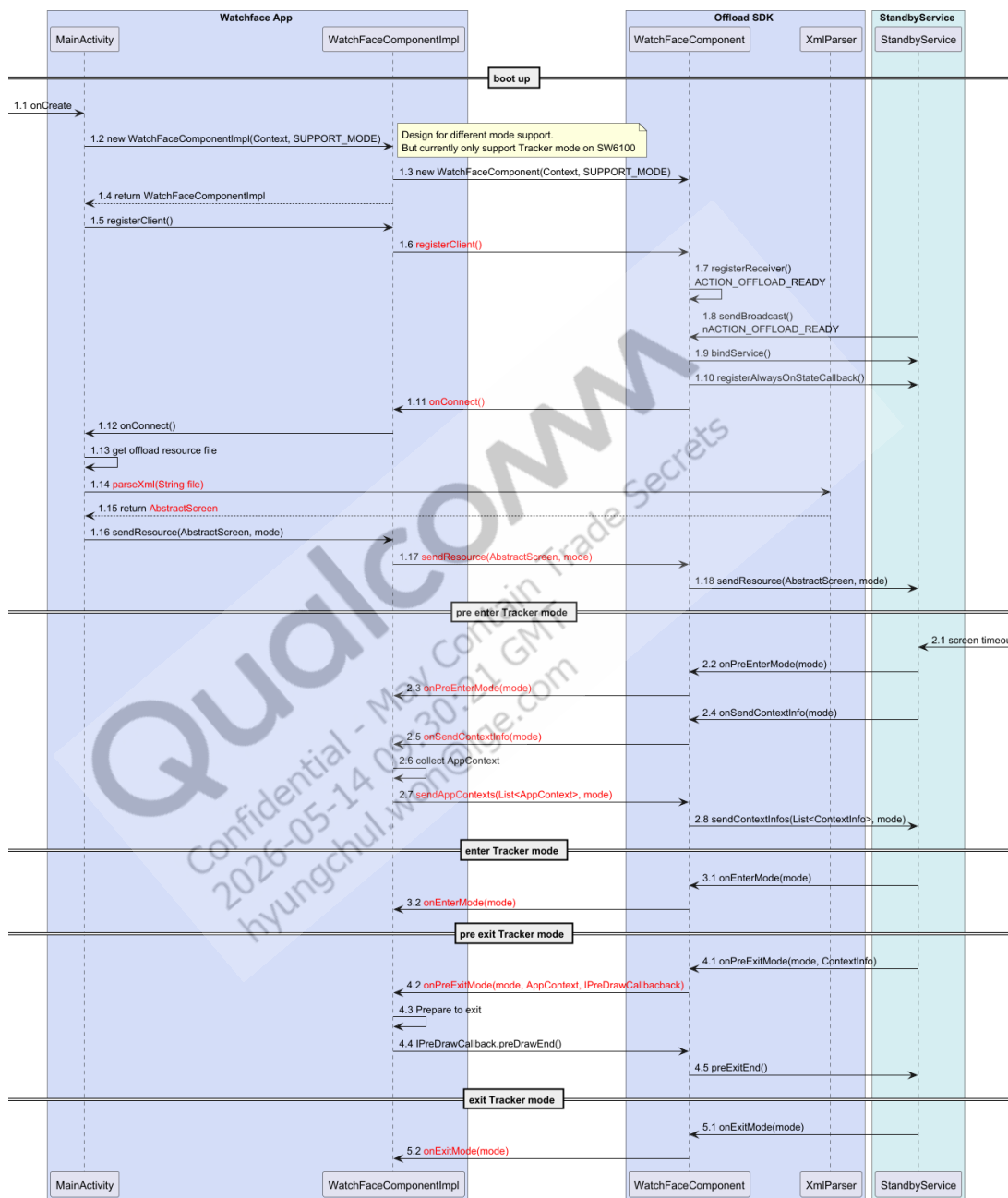


Figure 3-2 WatchFaceComponent usage core flow

3.2.3.2 StatusBarComponent

The `StatusBarComponent` follows a similar process to `WatchFaceComponent`, preparing and sending resources once the app starts after the device boots up.

3.2.3.3 OffloadUIComponent

`OffloadUIComponent` is used by general offload applications. The major difference between it and `WatchfaceComponent` is that it does not need to send resources when it starts. It just needs to do so when it receives the `onSendResourceCallback`.

Figure 3-3 shows the typical usage flow of `OffloadUIComponent`.

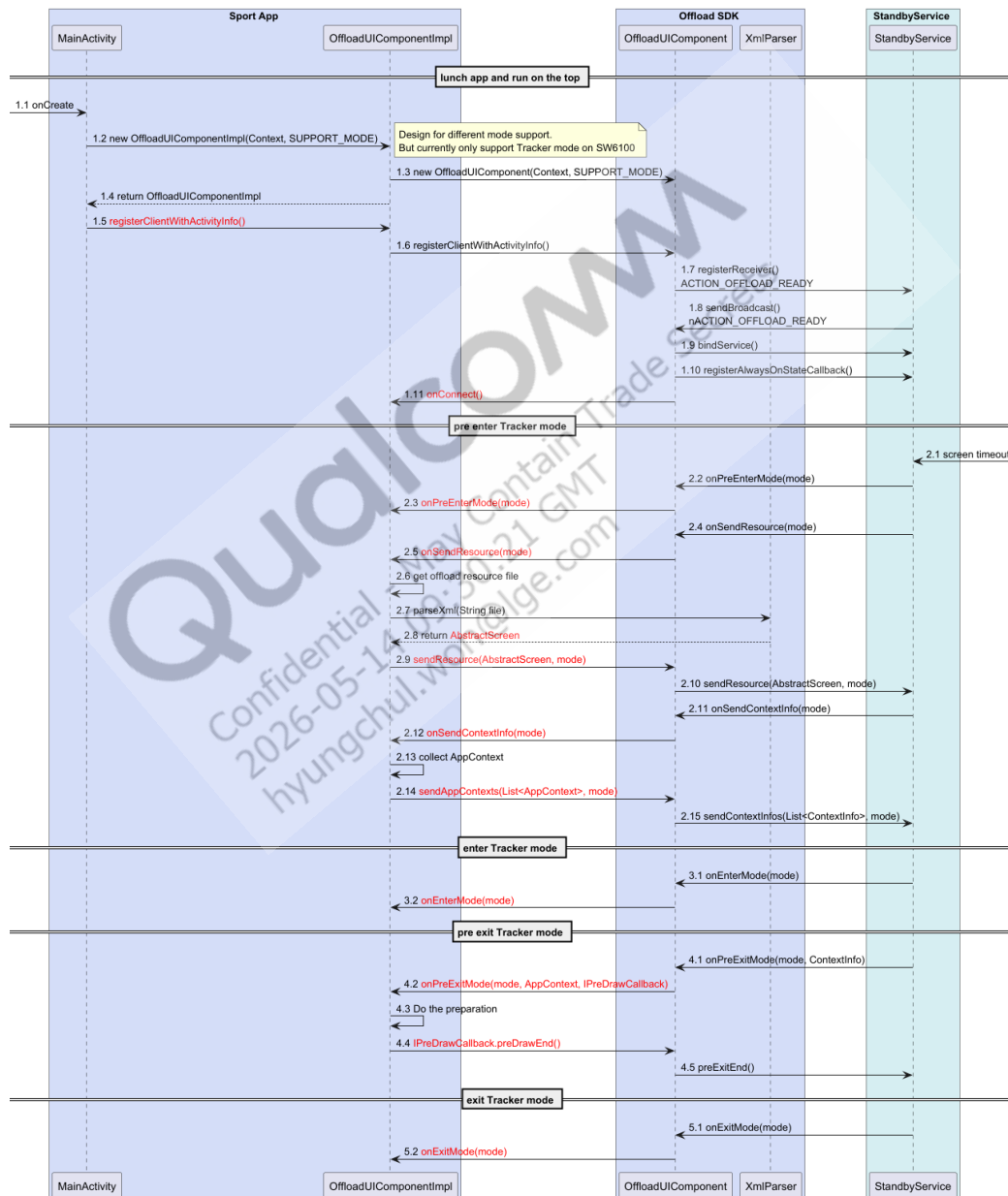


Figure 3-3 OffloadUIComponent usage core flow

4 Offload sample application

This chapter provides a development example of an Offload application. The example is a watchface Offload app that supports swipe-based transitions between the watchface screen and the sports screen. Both screens implement Offload capabilities, and the application ensures smooth visual transitions and consistent context when switching modes, demonstrating an end-to-end Offload design and implementation approach.

4.1 Prerequisites

Before developing an offload application, ensure that you have integrated the Offload SDK in your application as mentioned in [Section 2.2](#).

Next steps

- Prepare Offload resources: Prepare sample UI resource files
- Create and use Offload components: Shows how to implement and use Offload components.
- Bind and handle events: Shows how to implement, bind and handle events.

4.2 Prepare Offload resources

The sample application supports two screens, and their expected UI layouts are as follows:

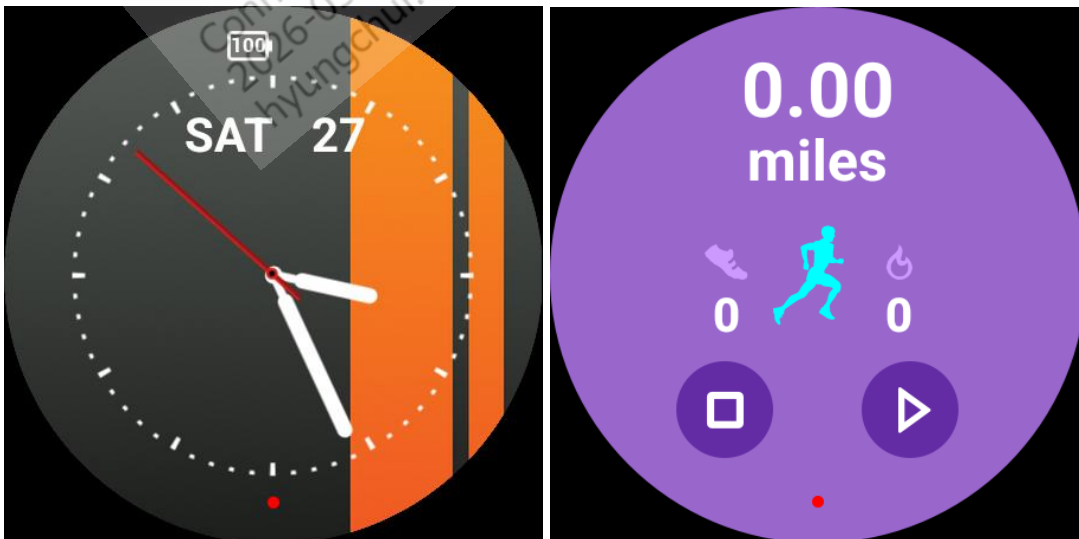


Figure 4-1 Sample Offload UI

NOTE: The battery indicator is part of the status bar content so is not included in this sample.

Based on the expected UI design, the reference Offload UI resource file are as follows:

/res/xml/sample_offload_layout.xml

```
<MultiScreen
    name="multi_screen"
    isActive="true"
    compositionStrategy="SCREEN_COMPOSITION">

    <Screen
        name="analog_watchface"
        compositionStrategy="LAYER_COMPOSITION"
        font="./fonts/OffloadNext-Regular.ttf"
        isActive="true">

        <Image
            name="background_image"
            offsetX="0"
            offsetY="0"
            width="454"
            height="454"
            needCompression="true"
            src="@drawable/bg_analog_load"/>

        <Text
            name="calendar_week"
            offsetX="152"
            offsetY="89"
            width="90"
            height="90"
            align="LEFT"
            color="#ffffffff"
            font="OffloadNext-Regular.ttf"
            fontSize="21"
            format="%s"
            textExpr="time.date.dayOfWeek"

        <Text
            name="calendar_date"
            width="90"
            height="90"
            offsetX="260"
            offsetY="89"
            align="RIGHT"
            color="#ffffffff"
            font="OffloadNext-Regular.ttf"
            fontSize="21"
            format="%02d"
            textExpr="time.date.day" />

    </Screen>

</MultiScreen>
```

```

        name="analog_hands_layout"
        offsetX="0"
        offsetY="0"
        width="454"
        height="454">
        <RotationLayout
            name="hands_rotation"
            pivotX="226"
            pivotY="225">
            <Image
                name="hour_hand_image"
                angleExpr="time.hour12.deg"
                offsetX="219"
                offsetY="136"
                src="@drawable/hour_hand" />

            <Image
                name="min_hand_image"
                angleExpr="time.min.deg"
                offsetX="219"
                offsetY="75"
                src="@drawable/min_hand" />

            <Image
                name="sec_hand_image"
                angleExpr="time.sec.deg"
                offsetX="221"
                offsetY="71"
                src="@drawable/sec_hand" />

        </RotationLayout>
    </RelativeLayout>
</Screen>

<Screen
    name="sports_face"
    compositionStrategy="NOT_COMPOSITION"
    font="./fonts/OffloadNext-Regular.ttf"
    isActive="true">

    <Text
        width="120"
        height="86"
        offsetX="97"
        offsetY="90"
        name="miles"
        align="CENTER"
        color="#ffffffff"
        font="OffloadNext-Regular.ttf"
        fontSize="30"
        format="%.2f"

```

```
        textExpr="qcom.sensor.distance.miles" />

<Text
    name="miles_m"
    width="150"
    height="86"
    offsetX="227"
    offsetY="90"
    align="CENTER"
    color="#ffffffff"
    font="OffloadNext-Regular.ttf"
    fontSize="30"
    format="%s"
    text="miles" />

<Image
    name="sport_step_image"
    width="59"
    height="48"
    offsetX="104"
    offsetY="172"
    src="@drawable/sport_step" />

<Image
    name="sport_calory_image"
    width="44"
    height="61"
    offsetX="317"
    offsetY="165"
    src="@drawable/sport_calory" />

<Image
    name="sport_man_image"
    width="60"
    height="85"
    offsetX="196"
    offsetY="188"
    needCompression="true"
    src="@drawable/sport_man_small" />

<Text
    name="step_data"
    width="60"
    height="61"
    offsetX="112"
    offsetY="248"
    align="CENTER"
    color="#ffffffff"
    font="OffloadNext-Regular.ttf"
    fontSize="22"
    format="%d"
```

```
        textExpr="qcom.sensor.step_count" />

<Text
    name="calory_data"
    width="60"
    height="61"
    offsetX="333"
    offsetY="248"
    align="CENTER"
    color="#ffffffff"
    font="OffloadNext-Regular.ttf"
    fontSize="22"
    format="%d"
    textExpr="qcom.sensor.calories" />

<Image
    name="mainbtn_stop"
    width="82"
    height="82"
    offsetX="106"
    offsetY="322"
    src="@drawable/mainbtn_stop">
    <EventParams
        eventDestType="TYPE_DEST_APPS"
        eventType="TYPE_CLICK" />
</Image>

<Image
    name="mainbtn_play"
    width="82"
    height="82"
    offsetX="263"
    offsetY="322"
    src="@drawable/mainbtn_play">
    <EventParams
        eventDestType="TYPE_DEST_APPS"
        eventType="TYPE_CLICK" />
</Image>

<Text
    name="btn_exit_tracker"
    width="90"
    height="90"
    offsetX="285"
    offsetY="20"
    align="CENTER"
    color="#ffffffff"
    font="OffloadNext-Regular.ttf"
    fontSize="25"
    format="%s"
    text="X">
```

```

        <EventParams
            wakeCapable="true"
            eventDestType="TYPE_DEST_APPS"
            eventType="TYPE_CLICK" />
    </Text>
</Screen>
</MultiScreen>

```

4.3 Create and use Offload components

This example creates a new class that extends and implements `WatchFaceComponent`:

```

package com.qualcomm.qti.offloadsample;

import android.content.Context;
import android.os.SystemProperties;
import com.qualcomm.qti.offload.AlwaysOnConstants;
import com.qualcomm.qti.offload.component.WatchFaceComponent;
import com.qualcomm.qti.offload.component.IPreDrawCallBack;
import com.qualcomm.qti.offload.resource.AbstractScreen;
import com.qualcomm.qti.offload.resource.interface.AbstractOffloadResource;

public class WatchFaceComponentImpl extends WatchFaceComponent {

    private final Context mContext;
    private int mSupportMode;
    private AbstractScreen mOffloadResource;

    public WatchFaceComponentImpl(Context context, int supportMode) {
        super(context, supportMode);
        mContext = context;
        mSupportMode = supportMode;
    }

    @Override
    public void onConnect() {
        // Send resource when connected.
        sendOffloadResources();
    }

    @Override
    public void onDisconnect() {}

    @Override
    public void onEnterMode(int mode) {}

```

```

@Override
public void onExitMode(int mode) {}

@Override
public void onPreEnterMode(int mode) {}

@Override
public void onPreExitMode(int mode, AppContext context,
                           IPreDrawCallback callback) {
    // Get current screen tile from AppContext and determine which
    // screen to show to keep seamless transition.
    // Here we debug with the log print as sample.
    // Actually, we need to pass the screen info to UI module to show
    // the right screen.
    String screenName = "analog_watchface";
    if (context != null) {
        ScreenContext sc = (ScreenContext) context;
        screenName = sc.getScreenName.
    }
    Log.d("need to show screen: " + screenName);
    // Need to call end when finish.
    callback.preDrawEnd();
}

@Override
public void onSendContextInfo(int mode) {
    // Collect current context info, such screen tile.
    // It will inform companion processor which screen to display to
    // keep seamless transition.
    // Here we only use a constant as sample. Actually, we need to
    // implement a proper way to manage multiple screens and get a
    // right screen name according to the current UI.
    List<AppContext> contexts = new ArrayList<>();
    ScreenContext screenContext = new
        ScreenContext("analog_watchface");
    contexts.add(screenContext);
    super.sendAppContexts(appContexts, mode);
}

@Override
public void onSendResouce(int mode) {}

public AbstractOffloadResource getOffloadResource() {
    // AbstractScreen is defined as a sub-class of
    // AbstractOffloadResource.

```

```

        return mOffloadResource;
    }

    private void sendOffloadResources() {
        String resourceFileName = mContext.getResources().getResourceName(
            R.xml.sample_offload_layout);
        mOffloadResource = super.parseXml(resourceFileName);
        super.sendResource(mSupportMode, mOffloadResource);
    }
}

```

After defining this component class to receive lifecycle callbacks from the Offload framework, it must be properly initialized and registered with the Offload framework at the appropriate stage.

Here is an example:

```

package com.qualcomm.qti.offloadsample;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import com.qualcomm.qti.offload.AlwaysOnConstants;
import com.qualcomm.qti.offloadsample.WatchFaceComponentImpl;

public class MainActivity extends AppCompatActivity {

    private WatchFaceComponentImpl mWatchFaceComponentImpl;
    private final int SUPPORT_MODE = AlwaysOnConstants.MODE_TRACKER;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Initialize offload component.
        mWatchFaceComponentImpl = new WatchFaceComponentImpl(
            this, SUPPORT_MODE);

        // Register to offload framework.
        mWatchFaceComponentImpl.registerClient();
    }
}

```

WatchFaceComponentImpl can be initialized once the app is started when the device boots up, it will proactively register itself and send its resources to the system.

4.4 Bind and handle events

As shown in earlier UI resource definitions, certain widgets on the **Sports** screen support click events. This information shows how to implement these events, extending `MainActivity` as an example.

```
package com.qualcomm.qti.offloadsample;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import com.qualcomm.qti.offload.AlwaysOnConstants;
import com.qualcomm.qti.offloadsample.WatchFaceComponentImpl;
import com.qualcomm.qti.offload.resource.interfaces.AbstractOffloadResource;
import com.qualcomm.qti.offload.resource.interfaces.IAppUxEventListener;
import com.qualcomm.qti.offload.resource.uxevent.AppUxEvent;
import com.qualcomm.qti.offload.resource.uxevent.UxEventType;

public class MainActivity extends AppCompatActivity
    implements AppUxEventListener {

    private WatchFaceComponentImpl mWatchFaceComponentImpl;
    private final int SUPPORT_MODE = AlwaysOnConstants.MODE_TRACKER;

    // offload resource.
    private final String SPORTS_PLAY_BUTTON = "mainbtn_play";
    private AbstractOffloadResource mSportsPlayBtn;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        .....// initialize work in 4.2

        // init UxEvent handler
        initUxEventHandler();
    }

    @Override
    public void onAppUxEvent(
        AbstractOffloadResource abstractOffloadResource,
        AppUxEvent appUxEvent) {
        // Here we only provide a sample to receive event from play
        // button. Developer may need to design a mechanism to manage
        // multiple widgets.
        String resourceName = resource.getName();
        if (resourceName.equals(SPORTS_PLAY_BUTTON)) {
            Log.d("receive uxevent from sport play button");
            if (appUxEvent.getEventType() == UxEventType.TYPE_CLICK) {
                // TODO
                // Response to the event according requirements.
            }
        }
    }
}
```

```
    }  
    }  
}  
  
private void initUxEventHandler() {  
    // Get global xml resource  
    AbstractOffloadResource offloadResource =  
        mWatchFaceComponentImpl.getOffloadResource();  
  
    // Get the specific resource by name  
    mSportsPlayBtn=offloadResource.findResourceByName(  
                                                SPORTS_PLAY_BUTTON);  
  
    if (mSportsPlayBtn != null) {  
        // Register UxEvent listener  
        mSportsPlayBtn.registerAppUxEventListener(this);  
    }  
}
```

From the sample code above, we can see the core of handling `uxevents` includes two steps:

1. Extend `IAppUxEventListener` to receive `AppUxEvent` from low layer.
2. Define and bind widgets to deal with the event.

This sample application only contains one specific widget. Developers need to design an effective mechanism such as annotation to manage if there are multiple interactive widgets.

A References

A.1 Data source set

Data source name	Elements	Attributes	Description
time.min	Text	textExpr	Indicates the minute of time.
time.hour12	Text	textExpr	Indicates the time of the 12-hour format.
time.hour24	Text	textExpr	Indicates the time of the 24-hour format.
time.date.day	Text	textExpr	Indicates the date.
time.date.dayOfWeek	Text	textExpr	Indicates the weekday.
time.sec.deg	RelativeLayout	angleExpr	Rotate at a specific angular speed of the second hand.
time.min.deg	RelativeLayout	angleExpr	Rotate at a specific angular speed of the minute hand.
time.hour12.deg	RelativeLayout	angleExpr	Rotate at a specific angular speed of the hour hand.
qcom.sensor.calories	Text	textExpr	Indicates the calories consumed.
qcom.sensor.step_count	Text	textExpr	Indicates the step counts.
qcom.sensor.distance.miles	Text	textExpr	Indicates the distance in miles.
battery.charging.level	Text	textExpr	Indicates the number according to the battery level.
dot.notification	Image	visibleExpr	Shows a dot presenting a notification.

A.2 Related documents

Title	Number
Qualcomm Technologies, Inc.	
SW6100.LAW.1.0 Offload SDK API Reference	80-74841-127

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-14 09:30:21 GMT
hyungchul.won@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("Qualcomm Technologies"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "Qualcomm Internal Use Only", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.